

Lab exercises

Part 12 —  AI Governance · 12-ai-governance/lab/EXERCISES.md

Lab — AI Governance

Work top to bottom. Each exercise has a **stated deliverable**; if you can't produce it, you haven't finished. Solutions are intentionally not provided — the checks tell you when you're right. Bring a real or realistic AI use case from your own organization if you can; the exercises work with an invented one too.

1. Warm-up: read the tool you were given (30 min)

Run the risk register against the demo portfolio:

```
/Users/srinip/ai-all/.venv/bin/python ../code/ai_risk_register_template.py --demo --legend
```

- **1a.** Without looking at the source, predict which of the six use cases will score highest and lowest. Now run it. Were you right? If not, which attribute surprised you?
- **1b.** The medical-triage assistant and the resume-screening tool both land in the Critical tier despite being wildly different products. Name the one attribute they share that drives this, and explain in one sentence why that attribute is weighted at 0.30 instead of 0.20.
- **1c.** The marketing-copy generator and the customer FAQ chatbot both land in Limited, for different reasons. Read each one's score breakdown and state, in one sentence per use case, *why* each is Limited rather than Minimal.

Deliverable: answers to 1a-1c, five sentences total.

2. Tier your own use case (1h)

Pick a real (or realistic) AI use case — from your own job if you have one, or invent one that isn't already in the demo set (a chat-based internal HR policy assistant, an AI-generated code-review comment bot, an AI system that drafts insurance claim denials, etc.).

- **2a.** Score it honestly on all four attributes using the `SCALES` dict in the script as your rubric. Write one sentence of justification per attribute — "I scored autonomy=3 because..." A bare number with no justification is not defensible to a committee and won't pass this exercise.
- **2b.** Add it to a JSON file shaped like `AIUseCase` and run it through `--input yourfile.json`. Report its score and tier.
- **2c.** Now change exactly one attribute by one point (e.g. `decision_autonomy` from 2 to 3) and re-run. Report the new score. Does it cross a tier boundary? If yes, explain what that would mean in practice for

the review requirements the system would face. If no, explain why the boundary is far enough away that this one-point shift doesn't matter.

Checks: - Your JSON file validates (the script raises a clear `ValueError` if any attribute isn't an int 1-5 — trigger this once on purpose and read the error message). - Your tier assignment is defensible: could you say it out loud to a skeptical VP without hand-waving?

3. Break the register on purpose (45 min)

The weights (`WEIGHTS` in the script) sum to 1.0 by design, and there's an `assert` enforcing it.

- **3a.** Change `WEIGHTS["data_sensitivity"]` to 0.70 and rebalance the others so they still sum to 1.0, favoring data sensitivity heavily over autonomy and reversibility. Re-run `--demo`. What happens to the fraud-detection and resume-screening use cases relative to each other? Which one now looks *less* risky than it should, and why is that a bad governance outcome?
- **3b.** Put the weights back to the original values. Now instead, edit the `TIERS` thresholds so the Critical ceiling starts at 60 instead of 75. Re-run `--demo`. How many use cases move into Critical, and is your organization's governance committee realistically staffed to review that many Critical-tier systems at the cadence notes/02 §2 describes?

Deliverable: a short paragraph (4-6 sentences) explaining why weights and thresholds are "policy, not physics" — the comment already in the script's source — using your 3a/3b results as evidence.

4. Write a model card for your own use case (2h)

Using `code/model_card_template.md` as your structural template (not as boilerplate to copy verbatim), write a complete card for the use case you tiered in Exercise 2.

- Every section from the template must be present: system overview, intended use, out-of-scope uses, training/grounding data summary, evaluation results, known limitations, human oversight design, change log, governance sign-off.
- Your evaluation section must state a methodology for every number, even if the numbers are your best realistic estimate for an exercise (say so explicitly: "estimated, not measured" is honest; a bare number presented as fact is not).
- Your human oversight design section must pick one of the three postures (HITL / HOTL / human-in-command) from notes/02 §4 and justify the choice against the risk tier from Exercise 2 — a Critical-tier system proposing HOTL-only oversight should fail your own review.

Check: hand your card to someone else (or re-read it cold after a day). Can they tell, from the card alone, exactly what the system is allowed to do, what it's explicitly forbidden from doing, and who signed off? If any of those three requires them to ask you a follow-up question, the card is incomplete.

5. Design the operating model for a 200-person company (1.5h)

Your fictional employer has ~200 employees, no existing AI governance function, and just discovered that three different teams have independently wired LLM calls into production (a support chatbot, an internal sales-notes summarizer, and a marketing draft generator) with no review of any kind.

- **5a.** Using notes/02 §2's role table as a menu, decide which roles you'd actually stand up in month one versus which ones are premature for a company this size. You do not need a dedicated MRM function on day one — say what you'd do instead, concretely (who plays that role part-time, and what changes when you outgrow that).
- **5b.** Tier all three existing systems using the register script (invent reasonable attribute values). Assign a review cadence and a named-role sign-off to each, matching the tier table in notes/01 §5.
- **5c.** Write the one-paragraph charter for your AI governance committee: who's on it, how often it meets for portfolio review, and what the fast-track SLA is for a single new launch decision.

Deliverable: three tiered use-case entries (name, score, tier, sign-off role) plus the charter paragraph. No org chart diagrams needed — plain sentences naming actual people/roles are more useful and harder to fake.

6. Incident postmortem, governance-style (1h) — the capstone

Scenario: the customer FAQ chatbot from `code/model_card_template.md` gave a customer incorrect information about a refund eligibility window, the customer acted on it (delayed their refund request past the actual deadline), and they complained publicly on social media.

Write a governance-style incident review (not a generic engineering postmortem) that answers, in order, per notes/02 §7:

1. **Triage:** given this system's Limited tier, what response speed and sign-off level does this trigger? Would the answer change if this were the Critical-tier medical-triage assistant instead — say exactly how.
2. **Containment:** which layer is actually broken here (retrieval corpus, prompt, escalation threshold, or something else), and what's the fastest safe mitigation?
3. **Root cause, past "the model hallucinated":** use the model card's §4 (known corpus gaps) and §6 (known limitations) — does this incident match a limitation the card already disclosed, or is it a new failure mode nobody caught? That distinction changes what "root cause" actually means here.
4. **Feedback to the risk register and eval suite:** what specific test case gets added to the evaluation suite (§5 of the card) so this exact failure is caught automatically next time? Does this incident change the risk tier itself, or just the mitigation?
5. **Disclosure:** what would you need to check (pointer to Module 14 [14-ai-compliance] is fine — you don't need to answer the legal question, just identify that it's a question) before deciding whether this needs external disclosure?

Check: your review should read like it changes something concrete — a new eval case, a corpus fix, a tightened escalation rule, or a re-tiering decision. A postmortem that ends at "we told the model not to do that again" has not found a root cause and does not pass this exercise.